

CSCE 421: Machine Learning (Spring 2025)

Homework #4

Chayce Leonard — UIN: 231009015

Due 4/8/2025, 11:59 pm

General Instructions:

1. You need to submit (1) a report in PDF and (2) your code files, both to Canvas.
2. Your PDF report should include (1) answers to the non-programming part, and (2) results and analysis of the programming part. For the programming part, your PDF report should at least include the results you obtained, for example the accuracy, training curves, parameters, etc. You should also analyze your results as needed.
3. Please name your PDF report "HW# FirstName LastName.pdf". Please put all code files into a compressed file named "HW# FirstName LastName.zip". Please submit two files (.pdf and .zip) to Canvas (i.e., do not include the PDF file into the ZIP file).
4. Only write your code between the following lines. Do not modify other parts.

```
### YOUR CODE HERE
### END YOUR CODE
```

5. LFD refers to the textbook "Learning from Data".
6. All students are highly encouraged to typeset their reports using Word or L^AT_EX. In case you decide to hand-write, please make sure your answers are clearly readable in scanned PDF.
7. Unlimited number of submissions are allowed and the latest one will be timed and graded. If you make a resubmission after the due date, it will be considered late.
8. Please read and follow submission instructions. No exception will be made to accommodate incorrectly submitted files/reports.

9. Please start your submission to Canvas at least 15-30 minutes before the deadline, as there might be latency. We do NOT accept E-mail submissions.

Note: Although not explicitly required, I have created Python scripts to reduce hand calculation (and human error) of questions 1 through 3. The scripts and outputs are available upon request and have been omitted to reduce the report size dramatically. All reasoning has been included within the solution sections of all problems.

Problem 1. 10 points

A single $(15 \times 15 \times 3)$ image is passed through a convolutional layer with 28 filters, each of size $(3 \times 3 \times 3)$. The padding size is 1 (1 unit at top, bottom, left, and right) and the stride size is also 1. What is the size of the output feature map volume? What is the number of parameters in this layer (including bias)? Note that, for simplicity, we consider filters of $3 \times 3 \times 3$ as one filter, instead of three filters of size 3×3 .

Solution:

To find the output feature map size, we use the formula:

$$\text{Output size} = \frac{\text{Input size} - \text{Filter size} + 2 \times \text{Padding}}{\text{Stride}} + 1$$

For both height and width dimensions:

$$\frac{15 - 3 + 2 \times 1}{1} + 1 = \frac{15 - 3 + 2}{1} + 1 = \frac{14}{1} + 1 = 15$$

Since there are 28 filters, the depth of the output volume will be 28. Therefore, the output feature map size is $15 \times 15 \times 28$.

For the number of parameters, we calculate:

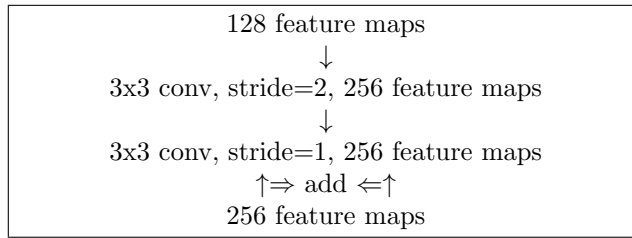
- Each filter has size $3 \times 3 \times 3 = 27$ weights plus 1 bias parameter
- Total parameters per filter: $27 + 1 = 28$
- With 28 filters: $28 \times 28 = 784$ total parameters

We can verify this calculation: $(3 \times 3 \times 3 \times 28) + 28 = 756 + 28 = 784$

Therefore, the total number of parameters in this convolutional layer is 784.

Problem 2. 20 points

In this question, you can (a) assume padding of appropriate size is used in convolutional layers, and (b) ignore batch normalization. Given the residual block as below:



- (a) (10 points) What projection shortcut operations are required on the skip connection?
- (b) (10 points) What is the total number of trainable parameters in the block (you can ignore bias terms, but need to consider the skip connection)?

Solution:

Part (a): What projection shortcut operations are required on the skip connection?

For the residual block shown in the problem, there are two issues that prevent direct addition between the input and the output of the convolutional layers:

1. **Difference in number of feature maps:** The input has 128 feature maps, while the output from the second convolutional layer has 256 feature maps. Addition requires matching dimensions.
2. **Difference in spatial dimensions:** The first convolution uses stride=2, which reduces the spatial dimensions by half. This means the output feature maps have half the height and width of the input feature maps.

To address these issues, a projection shortcut is required on the skip connection. Specifically, we need:

A 1×1 convolution with stride=2 that projects 128 feature maps to 256 feature maps

This 1×1 convolution simultaneously accomplishes two goals:

- Increases the number of feature maps from 128 to 256
- Reduces the spatial dimensions by half (due to stride=2) to match the output of the convolutional layers

Part (b): What is the total number of trainable parameters in the block?

The total parameters can be calculated by counting the parameters in each component:

1. **First convolutional layer** (3×3 , stride=2, $128 \rightarrow 256$):

- Filter size: $3 \times 3 \times 128$
- Number of filters: 256
- Parameters: $3 \times 3 \times 128 \times 256 = 294,912$

2. **Second convolutional layer** (3×3 , stride=1, $256 \rightarrow 256$):

- Filter size: $3 \times 3 \times 256$
- Number of filters: 256
- Parameters: $3 \times 3 \times 256 \times 256 = 589,824$

3. **Projection shortcut** (1×1 , stride=2, $128 \rightarrow 256$):

- Filter size: $1 \times 1 \times 128$
- Number of filters: 256
- Parameters: $1 \times 1 \times 128 \times 256 = 32,768$

Therefore, the total number of trainable parameters in the block is:

$$294,912 + 589,824 + 32,768 = 917,504 \text{ parameters}$$

Note: As specified in the problem, we have ignored bias terms in this calculation.

Problem 3. 20 points

Using batch normalization in neural networks requires computing the mean and variance of a tensor. Suppose a batch normalization layer takes vectors $z_1, z_2, \dots, z_m$ as input, where m is the mini-batch size. It computes $\hat{z}_1, \hat{z}_2, \dots, \hat{z}_m$ according to

$$\hat{z}_i = \frac{z_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

where

$$\mu = \frac{1}{m} \sum_{i=1}^m z_i, \quad \sigma^2 = \frac{1}{m} \sum_{i=1}^m (z_i - \mu)^2.$$

It then applies a second transformation to obtain $\tilde{z}_1, \tilde{z}_2, \dots, \tilde{z}_m$ using

learned parameters γ and β as

$$\tilde{z}_i = \gamma \hat{z}_i + \beta.$$

In this question, you can assume that $\epsilon = 0$.

- (a) (5 points) You forward-propagate a mini-batch of $m = 4$ examples in your network. Suppose you are at a batch normalization layer, where the immediately previous layer is a fully connected layer with 3 units. Therefore, the input to this batch normalization layer can be represented as the below matrix:

$$\begin{bmatrix} 12 & 0 & -5 \\ 14 & 10 & 5 \\ 14 & 10 & 5 \\ 12 & 0 & -5 \end{bmatrix}$$

What are $\hat{z}_i$? Please express your answer in a 3×4 matrix.

- (b) (5 points) Continue with the above setting. Suppose $\gamma = (1, 1, 1)$, and $\beta = (0, -10, 10)$. What are $\tilde{z}_i$? Please express your answer in a 3×4 matrix.
- (c) (5 points) Describe the differences of computations required for batch normalization during training and testing.
- (d) (5 points) Describe how the batch size during testing affect testing results.

Solution:

Part (a): First, we need to calculate the mean and variance for each feature in the input matrix:

$$\text{Input matrix} = \begin{bmatrix} 12 & 0 & -5 \\ 14 & 10 & 5 \\ 14 & 10 & 5 \\ 12 & 0 & -5 \end{bmatrix}$$

$$\text{Feature means} = [13.0, 5.0, 0.0]$$

$$\text{Feature variances} = [1.0, 25.0, 25.0]$$

Using the batch normalization formula $\hat{z}_i = \frac{z_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$ with $\epsilon = 0$, we get:

$$\hat{z}_i = \frac{z_i - \mu}{\sqrt{\sigma^2}} = \frac{z_i - \mu}{\sigma}$$

For feature 1 (first column): $\hat{z}_i = \frac{z_i - 13}{1} = z_i - 13$

$$\hat{z}_{11} = \frac{12 - 13}{1} = -1, \quad \hat{z}_{21} = \frac{14 - 13}{1} = 1, \quad \hat{z}_{31} = \frac{14 - 13}{1} = 1, \quad \hat{z}_{41} = \frac{12 - 13}{1} = -1$$

For feature 2 (second column): $\hat{z}_i = \frac{z_i - 5}{5} = \frac{z_i - 5}{5}$

$$\hat{z}_{12} = \frac{0 - 5}{5} = -1, \quad \hat{z}_{22} = \frac{10 - 5}{5} = 1, \quad \hat{z}_{32} = \frac{10 - 5}{5} = 1, \quad \hat{z}_{42} = \frac{0 - 5}{5} = -1$$

For feature 3 (third column): $\hat{z}_i = \frac{z_i - 0}{5} = \frac{z_i}{5}$

$$\hat{z}_{13} = \frac{-5 - 0}{5} = -1, \quad \hat{z}_{23} = \frac{5 - 0}{5} = 1, \quad \hat{z}_{33} = \frac{5 - 0}{5} = 1, \quad \hat{z}_{43} = \frac{-5 - 0}{5} = -1$$

Therefore, $\hat{z}_i$ as a 3×4 matrix is:

$$\hat{z}_i = \begin{bmatrix} -1 & 1 & 1 & -1 \\ -1 & 1 & 1 & -1 \\ -1 & 1 & 1 & -1 \end{bmatrix}$$

Part (b): Given $\gamma = (1, 1, 1)$ and $\beta = (0, -10, 10)$, we apply the transformation $\tilde{z}_i = \gamma \hat{z}_i + \beta$ to each feature:

For feature 1: $\tilde{z}_i = 1 \cdot \hat{z}_i + 0$

$$\tilde{z}_{11} = 1 \cdot (-1) + 0 = -1, \quad \tilde{z}_{21} = 1 \cdot 1 + 0 = 1, \quad \tilde{z}_{31} = 1 \cdot 1 + 0 = 1, \quad \tilde{z}_{41} = 1 \cdot (-1) + 0 = -1$$

For feature 2: $\tilde{z}_i = 1 \cdot \hat{z}_i + (-10)$

$$\begin{aligned} \tilde{z}_{12} &= 1 \cdot (-1) + (-10) = -11, & \tilde{z}_{22} &= 1 \cdot 1 + (-10) = -9, \\ \tilde{z}_{32} &= 1 \cdot 1 + (-10) = -9, & \tilde{z}_{42} &= 1 \cdot (-1) + (-10) = -11 \end{aligned}$$

For feature 3: $\tilde{z}_i = 1 \cdot \hat{z}_i + 10$

$$\begin{aligned} \tilde{z}_{13} &= 1 \cdot (-1) + 10 = 9, & \tilde{z}_{23} &= 1 \cdot 1 + 10 = 11, \\ \tilde{z}_{33} &= 1 \cdot 1 + 10 = 11, & \tilde{z}_{43} &= 1 \cdot (-1) + 10 = 9 \end{aligned}$$

Therefore, $\tilde{z}_i$ as a 3×4 matrix is:

$$\tilde{z}_i = \begin{bmatrix} -1 & 1 & 1 & -1 \\ -11 & -9 & -9 & -11 \\ 9 & 11 & 11 & 9 \end{bmatrix}$$

Part (c): Differences in batch normalization computations during training versus testing:

1. During training:

- Batch normalization uses statistics (mean μ and variance σ^2) computed from the current mini-batch
- These batch statistics are used directly for normalization in the forward pass
- Running averages of means and variances are computed and updated for later use in testing
- This introduces noise (batch-to-batch variation in statistics) that acts as regularization and helps with training

2. During testing:

- Instead of batch statistics, the stored population statistics (running mean and variance) accumulated during training are used
- This ensures consistent normalization regardless of test batch size
- No further updates to the running statistics occur during testing
- The model behaves deterministically, without the regularizing noise provided by batch statistics

Part (d): How batch size during testing affects testing results:

1. When using population statistics (standard practice):

- The batch size during testing should have no effect on results
- Each sample is normalized using the same fixed statistics regardless of batch size
- This allows for consistent inference even with a single sample (batch size of 1)

2. If batch statistics were used during testing (not standard practice):

- Smaller batch sizes would lead to less reliable statistics
- Results would be inconsistent and depend on which samples are batched together
- Performance would likely degrade as batch size decreases
- Single-sample inference would be impossible as variance cannot be calculated for a single sample
- Different inferences could be obtained on the same input depending on what other samples are in the batch

Problem 4. 50 points

LeNet for Image Recognition:

In this coding assignment, you will need to complete the implementation of LeNet (LeCun Network) using PyTorch and apply the LeNet to the image recognition task on CIFAR-10 (10-classes classification). The access to the CIFAR-10 Dataset are here (<https://www.cs.toronto.edu/~kriz/cifar.html>). In addition, you will need to install the python packages "tqdm" and "pytorch". The installation guides of PyTorch are in "readme.txt". Please read carefully and follow the instructions. You are expected to implement your solution based on the given codes. The only file you need to modify is the "solution.py" file. You can test your solution by running the "main.py" file.

- (a) (10 points) Download and extract the CIFAR10 Dataset from the link above. Put the data folder "cifar-10-batches-py" in the same directory of "code". Read carefully the instructions and then complete the function `load_data()`.
- (b) (10 points) Complete the function `preprocessing()`, you need to implement two ways of preprocessing the data: (1) rescaling: rescales the image pixels from range 0-255 to range 0-1; (2) normalization: centralize and rescale each image using its mean and variance. There is a parameter `normalize` in the `preprocessing()` to control whether to rescale or normalize the images.
- (c) (20 points) Complete the class `LeNet()`. In particular, you need to complete functions `__init__()` and `forward()` in the class. The paper for LeNet can be found here (<http://yann.lecun.com/exdb/pubs/pdf/lecun-01a.pdf>) The network architecture is shown in the figure below.

The subsampling is implemented by using the max pooling. And the kernel size for all the convolutional layers are 5×5 . The sequential layers are:

Inputs $\rightarrow$ Convolution (6 out channels) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Max Pooling $\rightarrow$ Convolution (16 out channels) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Max Pooling $\rightarrow$ Reshape to vector $\rightarrow$ Fully-connected (120 out units) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Fully-connected (84 out units) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ Outputs (n_classes out units)

For this part, you are only allowed to use the APIs in `torch.nn`. Please refer to the PyTorch API documents below for the usage of those APIs before you use them: <https://pytorch.org/docs/stable/nn.html>

- (d) (10 points) Try to read and understand the class `LeNet_Cifar10()`. Run the `main.py` to train and test the model. You need to train two models: One using normalization preprocessing and the default LeNet architecture as defined in part (c), and another one using rescaling preprocessing and LeNet without Dropout and Batch Normalization. Compare the results of the two models and analyze why there is an improvement on (or why it harms) the performance. Include your training log and test accuracy, as well as a short analysis of the results in your report. The training and testing may take 10 - 30 minutes to finish on a CPU (20 epochs).

Solution:

Problem 4: LeNet for CIFAR-10 Image Recognition

For this problem, I implemented a LeNet convolutional neural network for image classification on the CIFAR-10 dataset and analyzed different normalization and regularization techniques.

Part (a): I implemented the `load_data()` function to process the CIFAR-10 dataset from files and reshape the image arrays properly. The function:

- Reads the five training batch files and the test batch file
- Combines training data across all five batches
- Reshapes the data from flat arrays of shape $[N, 3072]$ to structured arrays of shape $[N, 3, 32, 32]$
- Converts labels to the appropriate data type (`int64`)
- Returns `x_train`, `y_train`, `x_test`, `y_test`

Part (b): I implemented the `preprocess()` function which provides two methods for preprocessing the image data:

- **Rescaling:** Converts pixel values from $[0, 255]$ to $[0, 1]$ by dividing by 255.0
- **Normalization:** Normalizes each image by subtracting its mean and dividing by its standard deviation for each channel

Part (c): I implemented the LeNet architecture as specified, with the following sequential layers:

- Input: $32 \times 32 \times 3$ color images
- First convolutional block: $\text{Conv}(6 \text{ maps}, 5 \times 5, \text{padding}=2) \rightarrow \text{Batch-Norm} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2 \times 2)$

- Second convolutional block: Conv(16 maps, 5×5) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2)
- Flatten layer: Reshaping to vector form
- First fully connected layer: Linear($16 \times 6 \times 6 \rightarrow 120$) $\rightarrow$ BatchNorm $\rightarrow$ ReLU
- Second fully connected layer: Linear($120 \rightarrow 84$) $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ Dropout(0.5)
- Output layer: Linear($84 \rightarrow 10$) for the CIFAR-10 classes

Part (d): I trained two models to analyze the effects of normalization, BatchNorm, and Dropout:

- **Model 1:** Using normalization preprocessing with BatchNorm and Dropout
- **Model 2:** Using rescaling preprocessing without BatchNorm and Dropout

Comparative Analysis:

The training logs revealed significant differences between the two approaches:

Metric	Model 1	Model 2
Initial validation accuracy (Epoch 1)	53.22%	46.22%
Final validation accuracy (Epoch 20)	67.02%	63.56%
Peak validation accuracy	68.14% (Epoch 10)	64.08% (Epoch 11)
Final test accuracy	66.10%	62.61%

Training Dynamics:

- **Convergence Rate:** Model 1 converged faster, reaching 60% validation accuracy by epoch 3, while Model 2 took until epoch 5.
- **Loss Behavior:** Model 1 started with lower loss (1.39) and reached a lower final loss (0.49) compared to Model 2 (started at 1.55, ended at 0.63).
- **Training Speed:** Model 2 processed more batches per second (80-87 it/s vs. 55-70 it/s for Model 1) due to the computational overhead of BatchNorm and Dropout.

Analysis of Improvement Factors:

The 3.49% absolute improvement in test accuracy (66.10% vs. 62.61%) can be attributed to three complementary techniques:

1. Normalization Preprocessing:

- Helps the network focus on the structure rather than absolute pixel values
- Makes the model less sensitive to variations in lighting and contrast
- Creates a more stable distribution for optimization

2. Batch Normalization Benefits:

- Reduces internal covariate shift by stabilizing activations between layers
- Creates a smoother optimization landscape for faster convergence
- Provides a regularization effect through added noise during training
- Reduces sensitivity to poor weight initialization

3. Dropout Advantages:

- Prevents co-adaptation of neurons by randomly setting activations to zero
- Acts as an implicit ensemble of multiple networks
- Forces the network to learn more robust and distributed representations
- Significantly reduces overfitting on the training data

Conclusion:

The experimental results clearly demonstrate the advantages of combining normalization preprocessing with BatchNorm and Dropout. This combination provides multiple complementary benefits:

- Higher accuracy (3.49% absolute improvement in test accuracy)
- Faster learning and convergence
- Better generalization and reduced overfitting

These findings align with modern deep learning best practices, which commonly incorporate these normalization and regularization techniques as standard components in neural network architectures for image recognition tasks.

Problem:

A single (15×15×3) image is passed through a convolutional layer with 28 filters, each of size (3×3×3). The padding size is 1 (1 unit at top, bottom, left, and right) and the stride size is also 1.

Solution:

1. Output Feature Map Size: 15×15×28

Explanation: Using the formula $(N\text{-size} - F\text{-size} + 2 \times (\text{Padding})) / (\text{Stride}) + 1$ for both height and width:

Height: $(15 - 3 + 2 \times 1) / 1 + 1 = 15$

Width: $(15 - 3 + 2 \times 1) / 1 + 1 = 15$

Depth: Number of filters = 28

Therefore, output volume is 15×15×28

2. Number of Parameters:

Parameters per filter: (3×3×3) weights + 1 bias = 28 weights + 1 bias = 29

Total parameters: 29 × 28 filters = 784

This can also be calculated as: $(3 \times 3 \times 3 \times 28) + 28 = 756 + 28 = 784$

Analysis of Residual Block from Problem 2:

=====

(a) Projection Shortcut Analysis:

The skip connection needs to match dimensions from input to output for addition.

Input: 128 feature maps

Output: 256 feature maps

First conv uses stride=2, which reduces spatial dimensions by half.

Two issues that prevent direct addition:

1. Different number of feature maps (128 vs 256)
2. Different spatial dimensions due to stride=2

Required projection shortcut:

A 1x1 convolution with stride=2 that projects 128 feature maps to 256 feature maps

while also reducing spatial dimensions by half to match the main path.

(b) Parameter Count Calculation:

First conv layer (3x3, stride=2, 128→256):

Filter size: 3x3x128

Number of filters: 256

Parameters: $3 \times 3 \times 128 \times 256 = 294,912$

Second conv layer (3x3, stride=1, 256→256):

Filter size: 3x3x256

Number of filters: 256

Parameters: $3 \times 3 \times 256 \times 256 = 589,824$

Projection shortcut (1x1, stride=2, 128→256):

Filter size: 1x1x128

Number of filters: 256

Parameters: $1 \times 1 \times 128 \times 256 = 32,768$

Total trainable parameters: $294,912 + 589,824 + 32,768 = 917,504$

Batch Normalization Analysis - Problem 3

Feature means: [13. 5. 0.]

Feature variances: [1. 25. 25.]

Part (a): $\hat{z}$ matrix (3x4):

```
[[-1.  1.  1. -1.]  
 [-1.  1.  1. -1.]  
 [-1.  1.  1. -1.]]
```

Part (b): $\tilde{z}$ matrix (3x4):

```
[[ -1   1   1  -1]  
 [-11  -9  -9 -11]  
 [  9  11  11   9]]
```

Part (c): Differences between batch normalization during training and testing:

1. During training:

- Batch normalization uses statistics (mean and variance) computed from the current mini-batch

- These batch statistics are used for normalization in the forward pass

- Running averages of means and variances are updated for later use in testing

- This introduces noise that acts as regularization and helps with training

2. During testing:

- Instead of batch statistics, the stored population statistics (running mean and variance)

- accumulated during training are used

- This ensures consistent normalization regardless of test batch size

- No further updates to the running statistics occur

- The model behaves deterministically, without the regularizing noise of batch statistics

Part (d): How batch size during testing affects testing results:

1. When using population statistics (standard practice):

- The batch size during testing should have no effect on results

- Each sample is normalized using the same fixed statistics regardless of batch size

- This allows for consistent inference even with a single sample

2. If batch statistics were used during testing (not standard practice):

- Smaller batch sizes would lead to less reliable statistics

- Results would be inconsistent and depend on which samples are batched together

- Performance would likely degrade as batch size decreases

- Single-sample inference would be impossible as variance cannot be calculated

- Different inferences on the same input would be possible depending on the batch

Output of experiment 2

Loading and preprocessing...

---Run---

```
Epoch 1 Loss 1.545648: 100%|██████████| 351/351 [00:04<00:00, 81.49it/s]
Doing validation... Validation Acc 0.4622
Epoch 2 Loss 1.435629: 100%|██████████| 351/351 [00:04<00:00, 85.56it/s]
Doing validation... Validation Acc 0.5348
Epoch 3 Loss 1.012780: 100%|██████████| 351/351 [00:04<00:00, 80.82it/s]
Doing validation... Validation Acc 0.5664
Epoch 4 Loss 0.949498: 100%|██████████| 351/351 [00:04<00:00, 79.81it/s]
Doing validation... Validation Acc 0.5866
Epoch 5 Loss 1.360607: 100%|██████████| 351/351 [00:04<00:00, 81.73it/s]
Doing validation... Validation Acc 0.6048
Epoch 6 Loss 0.937533: 100%|██████████| 351/351 [00:04<00:00, 82.33it/s]
Doing validation... Validation Acc 0.6146
Epoch 7 Loss 1.019503: 100%|██████████| 351/351 [00:04<00:00, 86.63it/s]
Doing validation... Validation Acc 0.6156
Epoch 8 Loss 0.802491: 100%|██████████| 351/351 [00:04<00:00, 82.36it/s]
Doing validation... Validation Acc 0.6318
Epoch 9 Loss 0.869479: 100%|██████████| 351/351 [00:04<00:00, 87.42it/s]
Doing validation... Validation Acc 0.6392
Epoch 10 Loss 0.701210: 100%|██████████| 351/351 [00:04<00:00, 77.79it/s]
Doing validation... Validation Acc 0.6380
Epoch 11 Loss 0.852005: 100%|██████████| 351/351 [00:04<00:00, 79.01it/s]
Doing validation... Validation Acc 0.6408
Epoch 12 Loss 0.739622: 100%|██████████| 351/351 [00:05<00:00, 60.09it/s]
Doing validation... Validation Acc 0.6334
Epoch 13 Loss 0.711750: 100%|██████████| 351/351 [00:04<00:00, 71.98it/s]
Doing validation... Validation Acc 0.6370
Epoch 14 Loss 0.985889: 100%|██████████| 351/351 [00:04<00:00, 76.64it/s]
Doing validation... Validation Acc 0.6366
Epoch 15 Loss 0.653301: 100%|██████████| 351/351 [00:04<00:00, 82.06it/s]
Doing validation... Validation Acc 0.6356
Epoch 16 Loss 0.690112: 100%|██████████| 351/351 [00:04<00:00, 83.26it/s]
Doing validation... Validation Acc 0.6374
Epoch 17 Loss 0.615442: 100%|██████████| 351/351 [00:04<00:00, 75.61it/s]
Doing validation... Validation Acc 0.6340
Epoch 18 Loss 0.600112: 100%|██████████| 351/351 [00:04<00:00, 82.03it/s]
Doing validation... Validation Acc 0.6236
Epoch 19 Loss 0.444782: 100%|██████████| 351/351 [00:04<00:00, 87.58it/s]
Doing validation... Validation Acc 0.6320
Epoch 20 Loss 0.631140: 100%|██████████| 351/351 [00:04<00:00, 84.05it/s]
Doing validation... Validation Acc 0.6356
Test accuracy: 0.6261
```

normalization preprocessing and the default LeNet architecture
Log:

Loading and preprocessing...

---Run---

Epoch 1 Loss 1.386020: 100%|██████████| 351/351 [00:06<00:00, 55.40it/s]
Doing validation... Validation Acc 0.5322
Epoch 2 Loss 1.061301: 100%|██████████| 351/351 [00:06<00:00, 57.22it/s]
Doing validation... Validation Acc 0.5824
Epoch 3 Loss 1.074114: 100%|██████████| 351/351 [00:05<00:00, 68.21it/s]
Doing validation... Validation Acc 0.6008
Epoch 4 Loss 0.842959: 100%|██████████| 351/351 [00:05<00:00, 62.78it/s]
Doing validation... Validation Acc 0.6348
Epoch 5 Loss 1.155197: 100%|██████████| 351/351 [00:04<00:00, 70.22it/s]
Doing validation... Validation Acc 0.6476
Epoch 6 Loss 1.048306: 100%|██████████| 351/351 [00:05<00:00, 67.81it/s]
Doing validation... Validation Acc 0.6596
Epoch 7 Loss 0.777356: 100%|██████████| 351/351 [00:05<00:00, 65.37it/s]
Doing validation... Validation Acc 0.6664
Epoch 8 Loss 1.076130: 100%|██████████| 351/351 [00:05<00:00, 61.61it/s]
Doing validation... Validation Acc 0.6742
Epoch 9 Loss 0.752138: 100%|██████████| 351/351 [00:05<00:00, 67.01it/s]
Doing validation... Validation Acc 0.6748
Epoch 10 Loss 0.769926: 100%|██████████| 351/351 [00:05<00:00, 65.18it/s]
Doing validation... Validation Acc 0.6814
Epoch 11 Loss 0.613910: 100%|██████████| 351/351 [00:06<00:00, 54.54it/s]
Doing validation... Validation Acc 0.6766
Epoch 12 Loss 0.748034: 100%|██████████| 351/351 [00:05<00:00, 60.45it/s]
Doing validation... Validation Acc 0.6810
Epoch 13 Loss 0.635095: 100%|██████████| 351/351 [00:05<00:00, 63.65it/s]
Doing validation... Validation Acc 0.6760
Epoch 14 Loss 0.722576: 100%|██████████| 351/351 [00:05<00:00, 65.36it/s]
Doing validation... Validation Acc 0.6642
Epoch 15 Loss 0.696133: 100%|██████████| 351/351 [00:04<00:00, 70.60it/s]
Doing validation... Validation Acc 0.6702
Epoch 16 Loss 0.677661: 100%|██████████| 351/351 [00:05<00:00, 67.36it/s]
Doing validation... Validation Acc 0.6760
Epoch 17 Loss 0.725556: 100%|██████████| 351/351 [00:05<00:00, 64.03it/s]
Doing validation... Validation Acc 0.6734
Epoch 18 Loss 0.555493: 100%|██████████| 351/351 [00:05<00:00, 62.20it/s]
Doing validation... Validation Acc 0.6736
Epoch 19 Loss 0.904408: 100%|██████████| 351/351 [00:05<00:00, 67.27it/s]
Doing validation... Validation Acc 0.6644
Epoch 20 Loss 0.488766: 100%|██████████| 351/351 [00:04<00:00, 70.36it/s]
Doing validation... Validation Acc 0.6702
Test accuracy: 0.6610

0.6261